CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)
CSPIT — Department of Computer Engineering · ITUE203 Web Development Frameworks

Student Name	Dhruv Bhoi	**Student ID**	**D26CE154**
Academic Year	2026–27 (ODD)	**Semester**	3rd Semester, B.Tech CE

Practical 7: PHP Form Processing with Server-Side Validation and CSV/JSON File Storage

Practical Aim & Tools

Aim: Implement PHP form processing for contact and feedback submissions with server-side validation, data sanitization, CSRF protection, and persistent storage in JSON/CSV files.

Tools & Technologies: PHP 8.x, CSV File I/O (`fputcsv`), JSON Storage (`json_encode`), CSRF Protection

1. Server-Side Processing Architecture

Client-side validation is easily bypassed; therefore, robust server-side processing is mandatory for security. StudentHub implements two storage workflows:
1. Contact Us (`contact.php`): Validates user inquiries, strips HTML tags (`clean()`), checks email formats (`FILTER_VALIDATE_EMAIL`), verifies anti-CSRF session tokens, and appends records atomically to `data/contacts.csv` using `fputcsv()` with exclusive file locks (`LOCK_EX`).
2. Student Feedback (`feedback.php`): Collects user star ratings (1 to 5) and comments, validates data integrity, and stores entries in `data/feedback.json`. Recent feedback cards are dynamically read and rendered below the form.

2. Implementation Code: Contact Processing & Atomic CSV Write

Listing: contact.php — Input Sanitization, CSRF Verification, and Atomic File Append

```
// contact.php - Server-Side Processing
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!csrf_check()) {
        $errors['form'] = 'Security validation failed (Invalid CSRF token).';
    } else {
        $name = clean($_POST['name'] ?? '');
        $email = clean($_POST['email'] ?? '');
        $message = clean($_POST['message'] ?? '');

        if (strlen($name) < 2) $errors['name'] = 'Name must be at least 2 characters.';
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) $errors['email'] = 'Invalid email
address.';
        if (strlen($message) < 10) $errors['message'] = 'Message must be at least 10
characters.';
```

```
        if (!$errors) {
            $fp = fopen(BASE_PATH . '/data/contacts.csv', 'a');
            if ($fp && flock($fp, LOCK_EX)) {
                fputcsv($fp, [date('Y-m-d H:i:s'), $name, $email, $message,
$_SERVER['REMOTE_ADDR']]);
                flock($fp, LOCK_UN);
                fclose($fp);
                flash('success', 'Thank you! Your message has been recorded.');
                redirect('contact.php');
            }
        }
    }
}
```

3. Verification Screenshot: Feedback & Rating System

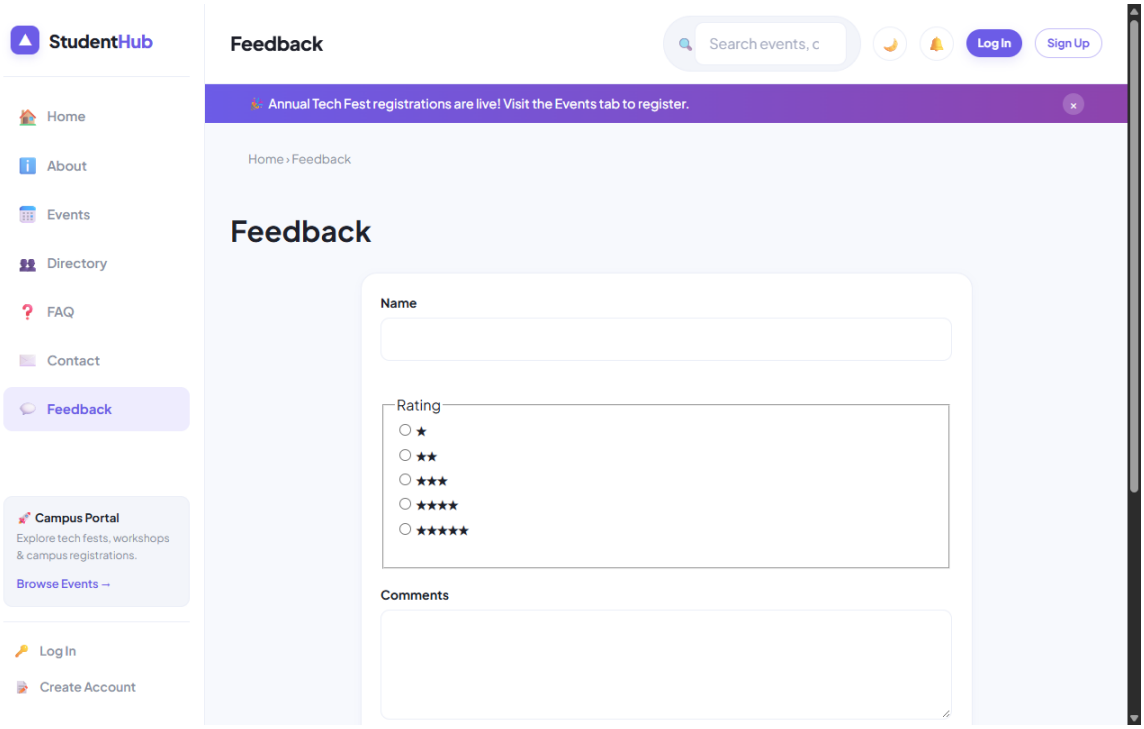


Figure: Feedback Submission Form with Star Ratings and Persisted JSON Feed

4. Verification Screenshot: Contact Inquiries Portal

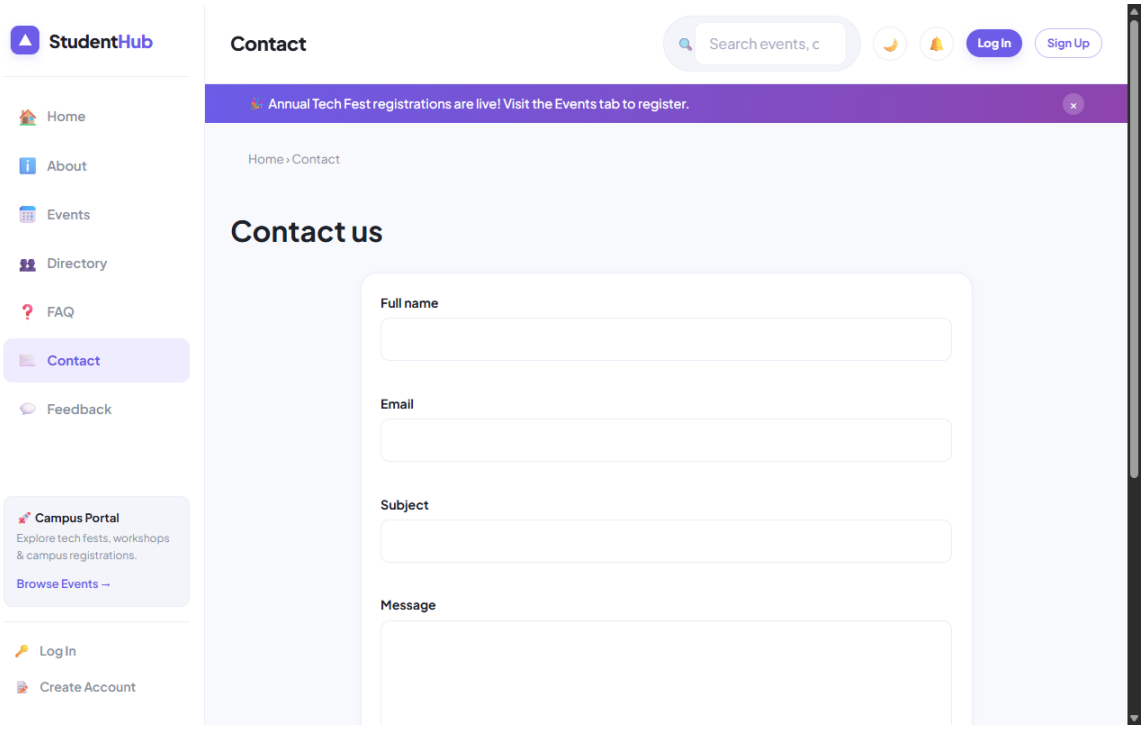


Figure: Contact Inquiries Form with Server-Side Validation and CSV Audit Logging

5. Key Questions & Analysis

Q: Why must form data be submitted using POST instead of GET?

Ans: GET appends parameters to the query string in browser history, server logs, and HTTP referrers, making it insecure for sensitive data. POST transmits data inside the HTTP message body, supports unlimited payload sizes, and complies with HTTP idempotency standards.

Q: How is file writing handled safely against concurrent race conditions?

Ans: By invoking `flock($fp, LOCK_EX)` before performing file writes. Exclusive locking prevents data corruption when multiple users submit forms simultaneously.

Practical 7 Conclusion & Learning Outcomes

Implemented secure server-side PHP form processing for contact inquiries and student feedback. Applied rigorous data sanitization, anti-CSRF token verification, and atomic file persistence using fputcsv with exclusive locks (LOCK_EX) and JSON encoding.